K215: Composing Checked Proofs and Testing Smaller Address Bounds

Exact source certificates, whole decoder paths,
native machine transport, and research toward 214 states

Research contribution prepared for Vladimir Reshetnikov's `ProveIt`

21 October 2026

Abstract

The accepted K215 machine already has complete Lean and Rocq correctness endpoints. The useful next step is to compose the smaller checked components and remove representation boundaries between them. This packet combines the incoming 1,426-entry normalized source certificate with verified balanced lookup, proves the exact all-register source equivalence, and preserves the public Rocq interface. Its indexing workload is 32,961 tree-node visits; the original 2,240-entry list certificate uses 5,018,706 visits under the same counting convention. These are exact indexing counts, not a whole-build speedup claim. Additional components connect the checked decoder blocks to whole paths of the literal source, and transport the existing CM215 alternative directly in the repository's stream-tape semantics. A separate address-allocation study proves an exact capacity criterion for ordered binary associations and constructs a source with a smaller maximum zero run while preserving both guarded control-flow graphs. The packet records the remaining physical allocation and endpoint-integration obligations. No correct machine with 214 or fewer states is claimed.

Repository pin. The source checkout is `aa66d0a33f1f8237a348899eb8095b045a502832`. The additional commits through `56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3` were reviewed. The maintained BusyBeaver Lean tree and the K225 Rocq tree are unchanged between those pins. Inputs and compiler receipts in this archive identify the exact bytes used.

Contents

1 The goal and the present proof boundary **3**
1.1 What the incoming proposals already established . . . 3

2 Normalized certificates with balanced lookup **4**
2.1 Exact statement and preserved interface . . . 4
2.2 Why the normalized certificate is legitimate . . . 4
2.3 A small refinement theorem composes the two optimizations . . . 4
2.4 Exact work counts and their interpretation . . . 5
2.5 Verification and adoption . . . 5

3 Source paths now generate the shared decoder proof **6**
3.1 Exact theorem and hypotheses . . . 6
3.2 How the proof composes . . . 6
3.3 Kernel binding to the upstream source . . . 7
3.4 Verification and remaining integration . . . 7

4 A checked CM215 transport in the accepted Rocq semantics **8**
4.1 Exact artifact and native statement . . . 8
4.2 A three-clause invariant replaces a full runtime proof . . . 9
4.3 One positive simulation proves both halting directions . . . 9
4.4 Verification and the remaining endpoint boundary . . . 9

5 A different state-reduction direction: bound address runs before shortening the scanner **10**
5.1 An exact alphabetic capacity criterion . . . 10
5.2 The permitted source class, and its exact minimum zero run . . . 11
5.3 A compact verified source, and the remaining physical problem . . . 12

6 Which 215-state candidate is easier to prove? **13**

7 A practical integration sequence **13**

8 Further research questions **14**
8.1 Joint optimization of zero runs and sharing . . . 14
8.2 A verified decoder compiler at the maintained interface . . . 14
8.3 Runtime synthesis with abstract spare storage . . . 14
8.4 A source proof expressed by local transformations . . . 14
8.5 Internal formalization and robust verification budgets . . . 14

9 Reproduction, contents, and scope **15**

1 The goal and the present proof boundary

The target convention is a deterministic Turing machine with one two-way infinite binary tape, blank input, left and right moves only, and a separate halting state. Initialization is included in the count of working states. The desired mathematical statement is

$$\mathrm{ZFC} \vdash (\neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})).$$

For K215, the repository already supplies the ordinary mathematical argument and external proof-assistant theorems relating halting to the existence of a finite ZFC refutation. The accepted dense table has 215 working states and 430 transition rows, with SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

Its source has 202 distinct subtrees and 4,751 primitive occurrences. These counts describe different objects: source subtrees, occurrence paths, and physical machine states must be priced separately. The current maintained Lean endpoint closure contains 184 project modules; the original fresh full build used 204 Lean modules and 237 Rocq sources [1, 2, 3].

The endpoint composition has five substantive interfaces: dense-state renaming; blank initialization; binary execution versus source execution; source equivalence versus the calibrated enumerator; and the connection between numerical derivations and ZFC refutations. This packet changes proof presentations and adds relative transport results. It does not encode a new derivation inside ZFC's own proof syntax. It also does not prove unconditional nonhalting or the minimality of 215 states.

1.1 What the incoming proposals already established

The newest incoming proposals are essential inputs [4]. The report does not count their conclusions as new results of this packet. In particular:

- The maintained Lean proof already uses ordinary source bisimulation, one-direction certificate checking with symmetry, shared semantic lemmas, and shorter implication and axiom-routine proofs.
- The checked-simplifications packet already supplies a 1,426-entry normalized Rocq checker, its all-register source theorem, a compatible public facade, and local Lean decoder blocks with an arbitrary-tape framing theorem.
- The certificate-refinements packet already supplies balanced lookup for the older 2,240-entry certificate, a generic marker-variant checker, and concrete zero-credit scheduler results.
- The automatic-certificates packet supplies a source-correspondence generator whose output is checked in both proof lanes. The producer is outside the logical trust boundary.
- The affine-cores and quotient-detachment packets contain additional source/control abstractions and logical simplifications. None supplies an accepted 214-state machine.

The main new work is at the joins: combining normalization with indexed lookup; deriving decoder window premises from a whole literal source path; and using the repository's native tape semantics for an alternative machine. The address experiment addresses a distinct physical obstacle.

2 Normalized certificates with balanced lookup

2.1 Exact statement and preserved interface

The source programs are the unchanged repository definitions `Prog215.p215` and `DagPerm.dagPerm`. For every natural-valued register file R, the delivered theorem proves

$$\mathrm{rm_halts}(p215, ([], R)) \quad \longleftrightarrow \quad \mathrm{rm_halts}(dagPerm, ([], R)).$$

The new theorem is `p215_dagPerm_normalized_balanced`; the exported facade retains the established name and type:

```
K215F.Source215.p215_dagPerm :
  forall R, rm_halts p215 ([], R) <-> rm_halts dagPerm ([], R).
```

A separate typed client elaborates that interface. The source files are compared byte for byte with the pinned repository definitions. Their SHA-256 identities are recorded in `upstream_binding.json`.

2.2 Why the normalized certificate is legitimate

A stored instruction pointer is a finite bit list, while the interpreter reads an implicit infinite tail of false bits. Define

$$P \sim Q \quad \Longleftrightarrow \quad \forall i \geq 0,\ \mathrm{bit}(P, i) = \mathrm{bit}(Q, i),$$

where an index beyond the list length returns false. This identifies trailing false padding only. In particular it does not justify replacing a consumed leaf path by a shorter path when evaluating break or continue. The interpreter continues to use its actual consumed path; normalization is applied only to completed segment endpoints.

The incoming `NormalizedBisim` checker already proves this discipline sound. An entry has two pointer representatives and two successor indices. The checker follows increment-only segments on both programs, compares their register increment counts, and checks equivalence of their endpoints under $\sim$. At a decrement checkpoint it checks the tested register and both branches. The existing theorem `hn_ipeq` moves a halting computation between equivalent pointer representatives without changing its register file or step bound. A genuine decrement step supplies the strict descent needed in the simulation induction. Generic mirroring supplies the reverse implication.

Normalization reduces the concrete certificate from 2,240 to 1,426 entries: 1,425 decrement checkpoints and one halt checkpoint. This reduction and its soundness proof are inherited. The present addition combines it with balanced storage.

2.3 A small refinement theorem composes the two optimizations

A certificate tree has empty, leaf, and binary-node constructors. Every binary node stores the number of entries in its left child. Let $\mathrm{flat}(T)$ be its leaf list. A tree is valid when every stored left size agrees with the actual left subtree size. The inherited balanced library proves

$$\mathrm{lookup}(T, j) = \mathrm{nth_error}(\mathrm{flat}(T), j) \qquad (\mathrm{valid}(T)).$$

The proof is a structural induction: the left branch uses the left prefix of the flattened list, and the right branch subtracts the verified left size.

The new `NormalizedBalanced.v` uses that lookup in the normalized branch checker. Its core theorem is the exact Boolean equality

$$\mathrm{ncheckTree}(t_1, t_2, T) = \mathrm{ncheck}(t_1, t_2, \mathrm{flat}(T)), \qquad \mathrm{valid}(T). \tag{1}$$

To prove it, first replace one tree lookup by its flattened-list lookup. That gives branch equality. Case analysis on the two decoded instructions gives entry equality. Finally, induction on the certificate tree identifies its conjunction of leaf checks with the list's `forallb`. The normalized soundness theorem then applies unchanged.

Theorem 2.1 (Composed certificate soundness)**.** *If T satisfies its checked size invariant and the normalized tree checker accepts p_1, p_2, T, then for every register file R,*

$$\mathrm{rm_halts}(p_1, ([], R)) \leftrightarrow \mathrm{rm_halts}(p_2, ([], R)).$$

Proof. The Boolean validity test implies tree validity. Equation (1) converts the accepted tree check to the inherited normalized list check. Apply its one-sided equivalence theorem, which uses the previously proved mirror theorem for the converse. No certificate generator is a premise. □

The generic composition module is 2,381 bytes. The concrete theorem is a small application of this result to the checked tree. These sizes describe the new glue, not the whole proof library. The old normalized and balanced soundness libraries are retained with attribution.

2.4 Exact work counts and their interpretation

For each successor query, count the list or tree constructors visited to find its entry. Include the initial checkpoint query, and omit successor queries for the halt entry. The exact census is:

Certificate	Entries	Queries	Visits	Max. tree depth
Original list	2,240	4,479	5,018,706	–
Original balanced tree	2,240	4,479	54,514	13
Normalized list	1,426	2,851	2,033,059	–
Normalized balanced tree	1,426	2,851	32,961	12

The first two rows are inherited from the certificate-refinements packet, whose `balanced_lookup/evidence/lookup_metrics.json` is copied into this component's `verification/inherited_old_certificate/` directory. The included `lookup_metrics.py` independently recomputes the last two rows; it does not claim to replay the older input. Thus the composed presentation uses about 39.54% fewer indexing visits than the older balanced certificate. The combined normalization and indexing reduce indexing visits by a factor of about 152.26 relative to the original list. The new count is also proved in Coq from the literal program and tree; the accounting function classifies each checkpoint using the actual `tsite` function.

These figures do not include instruction decoding, segment execution, path comparisons, tree construction, type elaboration, or kernel normalization of proof terms. In particular the ratio is not a runtime speedup. The source tree and its exact flattening are checked separately. The height theorem bounds every lookup, including an out-of-range one, by 12 visited nodes.

For a balanced N-entry certificate issuing $O(N)$ queries, this indexing component takes $O(N \log N)$ node visits. For in-range indices, or more generally indices represented with $O(\log N)$ bits, charging linear cost for binary comparison and subtraction gives the conservative $O(N \log^2 N)$ bit-operation bound for indexing. Other checker components retain their existing costs.

2.5 Verification and adoption

The proof is compiled with Coq 8.18.0, using the inherited compatibility lemmas for names from the repository's Rocq 9.1 lane. Those compatibility lemmas are proved, and the selected assumption

reports are closed. The source theorem and typed public facade are kernel-checked. The separate concrete recursive audit and its exact receipt are included with the component.

The production adoption step is to keep the established public interface, add the composed checker to the source-generation/build manifest, and rebuild `Correct215` and `CorrectDense215` under the maintained Rocq toolchain. The complete 237-source machine-to-ZFC closure is not rebuilt by this source-layer experiment. The automatically inferred certificate can be fed to the same checker; soundness depends only on acceptance, not on how its data was produced.

3 Source paths now generate the shared decoder proof

The incoming checked-decoder packet already supplied a sound finite-window executor, a theorem composing finite chains of restored windows, and a concrete certificate for K215. Its concrete data comprise 281 distinct local blocks, 234 context roles, 371 internal role edges, and 97 primitive-entry edges. Those data and their three Lean modules are reused unchanged here. The remaining gap was semantic: a caller still had to supply every window premise of a `Chain`, and the identification of the certificate's source with the upstream source had only been checked outside the kernel.

The new component closes both gaps. It proves that the actual source path provides all the window facts, and it identifies the certificate's expanded source with the literal upstream source by kernel reduction. It thus promotes the shared decoder construction from local block soundness to a theorem about complete decoder traversal of the repository's source program.

3.1 Exact theorem and hypotheses

Write $\mathrm{Walk}_{215}(P) = (e, d)$ for the repository's `RegisterPath.walk` on `KSource215.p215`, with the same `nnodes` + 1 fuel and root used upstream. Let $Q = \mathrm{consume}(P, d)$, so $|Q| = d$; this operation incorporates both implicit zero padding and truncation of an instruction-pointer input. Let $t : \mathbb{Z} \to \{0, 1\}$ be an arbitrary complete tape, and let a be the initial head position. Assume

$$t(a-1) = 1, \qquad t(a+i) = Q_i \quad (0 \le i < d).$$

No condition is imposed on the other tape cells. The theorem `DecoderSourceBinding.upstream_walk_reaches` proves the existence of an execution length N, an entry state q, a scanned bit b, and the matching primitive operation and argument, such that

$$d - 1 \le N \le 7(d-1), \qquad \mathrm{run}^{N}_{K215}(84, a, t) = (q, a + d - 1, t).$$

In particular the entire tape is restored, including its unrestricted exterior. The final bit is $b = t(a + d - 1)$. A certified role binds this state and bit to the exact source primitive e and to the literal K215 action selected there. The execution ends just before that primitive action is performed.

The result is uniform in P, e, d, a, and t. It is not limited to the blank-tape startup or a finite collection of sampled runs. Its source-word hypothesis is the natural premise at a decoder entry; the existing register representation and cleanliness invariant are responsible for establishing that premise when this result is connected to the whole interpreter.

3.2 How the proof composes

The proof introduces two elementary tape predicates. `Reads` t a w says that the list w occurs starting at position a. `Before` t a s says that the context s ends immediately before a. The generic list lemmas show that appending the currently scanned bit extends this context, taking a suffix

preserves its right alignment, and a right-aligned context implies the overlay equality required by a local block. Thus a block's finite-window hypothesis is derived from the source word itself.

A role stores a source node, a decoder state, and a context of at most two preceding bits. The source-path induction has two cases. For an internal edge, the structural certificate identifies the child role, its updated context, and the block to execute. The just-proved tape lemmas establish that block's window premise. The induction hypothesis supplies the child's chain, and `Chain.more` prepends the certified block. For a primitive edge, zero internal steps remain; the certificate provides the exact native transition row and its source leaf. This proves `source_path_chain`. Applying the incoming `chain_sound` theorem gives the full-tape execution.

There is no duplicate concrete structural check. The generic theorem `edgeSpec_of_check` extracts the semantic facts from the existing Boolean `edgeCheck`. The theorem `edges_of_structure` then derives them for every role from the already checked `K215Decoder.structure_checked`. The inherited `all_blocks_checked` theorem supplies all local execution facts. A small additional finite duration check verifies that every block takes between one and seven steps. Induction on a chain proves the displayed bounds, since each internal block advances the restored head by exactly one source bit.

3.3 Kernel binding to the upstream source

The inherited certificate represents its source as 202 nodes: 179 internal binary nodes and 23 distinct primitive leaves. The upstream register source uses a 179-node DAG whose edges contain the primitive operations directly. Their numbering conventions therefore differ. The adapter defines an explicit expansion of the certificate nodes into the upstream `RMBisim.Tree` type, translating the five operation tags into the corresponding `RegisterPath.Edge` constructors.

The exact identity

$$\mathrm{sourceTree}(\texttt{sourceNodes}, 203, \texttt{sourceRoot}) = \texttt{KSource215.t215}$$

is proved by `rfl`. It is a kernel-checked definitional equality, with no assumed correspondence and no external hash used as a proof premise. The generic theorem `source_path_of_leaf` converts every upstream `SiteCoverage.LeafAt` witness through this identity into a source-DAG path. Finally the existing `twalk_leafAt`, `walk_root`, and consumed-prefix equality connect the theorem to `RegisterPath.walk`. The pinned source has 4,751 primitive occurrences, with path depths from 3 to 47, as also recorded by the supplied source audit.

3.4 Verification and remaining integration

The portable component contains two new Lean modules, three unchanged incoming decoder modules, and ten unchanged upstream source modules. Its reproduction driver compiles this 15-module closure from source with official Lean 4.32.0; Mathlib and prebuilt repository objects are unnecessary. The source index identifies each inherited file and binds its bytes to repository commit `aa66d0a33f`. Compiler receipts and logs accompany the component. The delivered verification combines successful builds in the same session, with exact source, object, and imported-object hashes; the two new modules were recompiled against that closure. A subsequent single fresh replay was interrupted by an operating-system termination while rebuilding the already verified source-data module, so it is recorded as a resource failure rather than a successful complete replay. The literal source-tree identity has no axioms. Both the complete source-path and upstream-walk endpoints report only `propext`, `Classical.choice`, and `Quot.sound`; no admitted theorem or native evaluator is introduced.

This closes the source-path and source-expansion obligations of the incoming shared-decoder proposal. To replace `KSites215`, the next step is to compose this theorem with context-parametric primitive-operation and pointer return contracts. The existing 97 entry bindings identify the required interfaces; they do not by themselves prove those operations. An adapter between the native integer-tape presentation here and the repository's renamed K230-label zipper presentation must also be supplied. In particular, `KSim.Covered` currently requires a successful per-leaf Boolean check; a reusable semantic-coverage interface would allow the shared traversal and primitive contracts to feed its simulation theorem directly.

The benefit is a precise change in proof organization: internal decoding is proved through 234 reusable contexts rather than separately for 4,750 internal source occurrences. The full endpoint's compilation-time improvement remains to be measured after primitive and return integration. The machine itself is unchanged, and this decoder result does not constitute a 214-state construction.

4 A checked CM215 transport in the accepted Rocq semantics

The conditional-marker alternative CM215 was already present in the repository. The contribution here is a new connection to the accepted formal endpoint: its equivalence to K215 is now proved directly in the *same* Rocq state and tape types as `K215F.Dense215`. This avoids an additional representation theorem between an integer-indexed tape and the native coinductive streams. The result concerns the existing 215-state table; it is not a new 214-state construction.

4.1 Exact artifact and native statement

The packet retains the existing dense `marker215.tm`, with 215 working states and 430 transition rows, and generates its literal `CM215Table.v`. Its SHA-256 is

`1f4f084f7293e2b042d5746039f793ff72ed429083b504970c939a5515e666cf.`

The reference table and the four imported K215 semantics files are unmodified copies from commit `aa66d0a33f1f8237a348899eb8095b045a502832`. The code uses the accepted type of bounded states, `bnat 215`, and native configurations

$$c = (q, (\ell, b, r)), \qquad \ell, r : \mathrm{Stream}(\mathrm{Bool}).$$

Here b is the scanned bit, and the stream heads are the two adjacent bits. The physical position is implicit in this centered tape representation.

The newly compiled theorem `all_tapes_equihalting` states

$$\forall \ell, b, r, \quad \mathrm{Halts}_{\mathrm{Dense}}(K215, (0, (\ell, b, r))) \iff \mathrm{Halts}_{\mathrm{Dense}}(CM215, (0, (\ell, b, r))).$$

Thus both sides use the actual `Dense.halts` of the accepted Rocq development. The theorem quantifies all initial binary tapes at control zero, including tapes that do not encode registers. It does not assert an unconditional equivalence from arbitrary initial internal control states.

State	Read	K215 row	CM215 row
52	0	$0\,R\,149$	$0\,L\,170$
151	1	$1\,R\,170$	$1\,R\,52$
160	0	$0\,R\,170$	$0\,R\,52$
170	0	$0\,R\,163$	$0\,R\,178$
170	1	$1\,R\,145$	$1\,R\,125$
184	0	$0\,R\,170$	$0\,R\,52$

4.2 A three-clause invariant replaces a full runtime proof

Define $\phi(170) = 52$ and $\phi(q) = q$ otherwise, and let $F(q, t) = (\phi(q), t)$. The invariant on the *old* machine is

$$\begin{aligned} q = 52 &\implies \mathrm{hd}(\ell) = 1, \\ q = 170 \ \wedge \ b = 0 &\implies \mathrm{hd}(\ell) = 0, \\ q = 151 &\implies \mathrm{hd}(r) = 1. \end{aligned}$$

It is immediate at control zero. Its induction step follows from three finite incoming-row facts. Every transition into 52 writes one and moves right. Every transition into 151 writes one and moves left. A transition into 170 either writes zero and moves right, or comes from 151 and moves right. In the latter case the third invariant clause says that the new scanned bit is one, so the conditional clause at 170 has a false antecedent. This auxiliary clause is what turns the otherwise historical argument into an ordinary one-step invariant proof.

Outside 52 and 170, the transition tables commute with ϕ. At either special state with scanned bit one, both behaviors take one right-moving step to 145. With scanned bit zero, CM215 moves left to 170, reads the origin marker, moves right through 125 or 178, and takes one further right-moving step to 149 or 163. The three steps reproduce the old single step exactly, including the entire two-sided stream tape. The proof destructs a stream constructor where needed; it does not assume a general extensionality principle for streams or functions.

All finite row obligations, including halting reflection, are bundled into one decidable predicate and checked over all 216 bounded controls and both bits. This includes the distinguished terminal control. The certificate `checked_facts` is proved by kernel reduction; its computation took approximately 0.07 seconds in the development run. The generator and the Python input audit are not trusted: the theorem itself checks the literal Coq table and the certificate soundness.

4.3 One positive simulation proves both halting directions

The general lemma `positive_transport`, written over the native `Dense` semantics, assumes preservation of an invariant, agreement of halting at synchronized configurations, and a positive target path for every old step. A forward induction transports a finite old run. For the reverse direction, the existing BusyCoq theorem `progress_halts` extracts an old halting point from a terminating new run: an unbounded sequence of old steps would produce an unbounded sequence of positive new blocks. This uses no separately generated reverse certificate. The CM215 instance supplies blocks of length one or three.

The practical consequence is that CM215 needs neither a new blank-start replay nor a new proof of all decoder sites. The accepted K215 calibration, source proof, and symbolic runtime are reused through a theorem at their final machine semantics. This is a particularly economical way to certify local changes to an already accepted candidate.

4.4 Verification and the remaining endpoint boundary

Coq 8.18.0 compiled the nine-source reproduction closure, comprising the three pinned BusyCoq sources, four exact K215 table/semantics sources, and the two new CM215 files. Both `all_tapes_equihalting` and `blank_halts_iff` print *Closed under the global context.* The matching `coqchk` recursively rechecked `CM215Stream`. The inherited tactic library imports some unused axioms; the two target assumption reports, which follow their actual dependency terms, are closed.

The reproduction driver checks all input hashes and independently binds all 430 textual machine rows to the literal Coq table.

The accompanying `CM215Endpoint.v` is a short integration client composing `blank_halts_iff` with the existing `tm215_halts_iff_ZFC_inconsistent`. That client and the full inherited endpoint closure were not rebuilt here. Its source is explicitly labelled integration-only and excluded from the standalone reproduction command. The freshly verified contribution is the exact-semantic, all-tapes equivalence; the existing accepted K215 endpoint supplies the calibration when the client is integrated.

CM215 is therefore a good immediate alternative when proof reuse is the criterion. The related M124 marker construction has an even simpler local invariant, with two unconditional predecessor-bit clauses, and is a promising next instance of the same native transport interface. Neither choice removes the substantive inherited calibration from the initial correctness argument; each sharply limits what must be reproved for the changed physical table.

5 A different state-reduction direction: bound address runs before shortening the scanner

The source layout exposes a route that is distinct from merging physical states. The accepted K215 source has 202 distinct subtrees, 4,751 primitive occurrences, maximum address depth 47, and maximum consecutive-zero run six. The conditional-marker research packet already records a failed experiment that deletes native state 50 and redirects row 34/0 from $0L50$ to $0L75$. On the original source, 72 guarded cases are unproved at 37 addresses ending in six zeros; one has a concrete wrong-successor trace. Consequently, merely deleting that scanner state is not justified. Instead, first change the source association to eliminate six-zero runs, while preserving every primitive and guarded successor, and only then attempt the shorter scanner. This section supplies such a source transformation and an exact feasibility theorem for the permitted source grammar. It does not supply a successful 214-state allocation.

5.1 An exact alphabetic capacity criterion

Fix a maximum zero-run length Z. A closed source block has capacity c if it can be represented so that its addresses obey this limit even when entered after any run of at most c zeros. Capacity -1 means no admissible representation even with incoming run zero. This definition concerns all address prefixes: checking every leaf address is equivalent, since every internal prefix extends to a leaf. At a binary node, the left edge extends the incoming zero run by one and the right edge resets it to zero. Thus two fixed children with capacities a, b have capacity

$$a \star b = \begin{cases} a - 1, & a \geq 1 \text{ and } b \geq 0, \\ -1, & \text{otherwise.} \end{cases}$$

Theorem (alphabetic capacity). Let $c_1, \ldots, c_n$ be the nonnegative capacities of ordered atoms whose association is free, and let $r \geq 0$ be an incoming zero run. For $n = 1$, feasibility is exactly $r \leq c_1$. For $n \geq 2$, some full binary association of these atoms is feasible if and only if

$$\sum_{i=1}^{k} c_i - k \;\geq\; r \qquad (1 \leq k < n). \tag{2}$$

In particular, its greatest feasible incoming run is $\min_{1\le k<n}(\sum_{i=1}^{k} c_i - k)$ if this is nonnegative; otherwise there is no feasible association. If any atom has capacity -1, there is likewise no feasible association. The feasibility test and a witnessing association can be produced in linear time in n.

Proof. Use the standard preorder tree-slot encoding of a full ordered binary tree. Let d_i count the internal nodes opened immediately before its ith leaf. Equivalently, d_i is the number of consecutive left edges on the final part of that leaf's address. For the first leaf, its entering run is $r + d_1$; for every later leaf a preceding right edge has reset the run, so its entering run is d_i. The full-tree conditions are

$$\sum_{i=1}^{n} d_i = n - 1, \qquad \sum_{i=1}^{k} d_i \ge k \quad (1 \le k < n), \qquad d_i \ge 0.$$

Indeed, opening an internal node increases the number of pending child slots by one; consuming a leaf decreases it by one. The displayed prefix conditions say that at least one slot remains until the last leaf, when none remains. They also give the converse construction of a unique full binary tree from this preorder sequence.

Feasibility requires $d_1 \le c_1 - r$ and $d_i \le c_i$ for $i > 1$. Applying these bounds to each proper prefix proves (2). Conversely, assume those inequalities. Set $c'_1 = c_1 - r$ and $c'_i = c_i$ otherwise, and begin with budget $B = n - 1$. Successively choose $d_i = \min(c'_i, B)$ and replace B by $B - d_i$. The prefix inequalities imply that the first $n - 1$ capacities already cover the whole budget; hence $d_n = 0$ and B ends at zero. For every $k < n$, the greedy prefix sum is $\min(\sum_{i\le k} c'_i, n - 1) \ge k$. It therefore defines a full tree, and each atom receives an allowed entering run. A stack reads the resulting preorder sequence and constructs the tree in linear time. The one-atom case has no connecting edges and is immediate. This is an application of standard preorder/Łukasiewicz tree encoding, not a claim of novelty for that encoding [6]. □

The independent checker compares this formula with exhaustive Catalan recursion over every ordered capacity vector of lengths one through seven with entries in $\{-1, 0, 1, 2, 3\}$. All 97,655 vectors agree. For the 13,516 feasible vectors, every allowed incoming capacity is reconstructed and independently evaluated, totaling 25,190 constructive checks. These finite checks corroborate the proof above; the proof does not infer a universal claim from those tests.

5.2 The permitted source class, and its exact minimum zero run

The source language records each subtree's outgoing-control boundary as a triple. A factor is *closed* when this is $(0, 0, \text{false})$: there is no escaping forward or backward jump and no halt. The permitted rewrite is

$$A; (B; C) \quad \longleftrightarrow \quad (A; B); C \qquad \text{when } A, B, C \text{ are closed.}$$

These transformations preserve the ordered primitive word and all guarded successors, including both branches of a conditional decrement. They may be performed independently at arbitrary occurrences. Flatten each maximal association of closed factors into its ordered atomic sequence. Its atoms cannot be split by a permitted rotation at their root; recursive rotations inside an atom preserve its boundary. Therefore the capacity theorem, applied recursively to these sequences and the fixed remaining nodes, decides the entire closed-associativity class, not a bounded rotation beam.

For this K215 source the least possible maximum zero run in that class is **five**. There is a particularly small lower-bound certificate. At prefix `1101001001111`, the source contains a 13-leaf fragment spanning ordinals 4578–4590. Write D_j for $\mathrm{CD}(j)$, B_j for $\mathrm{CB}(j)$, and C_j for $\mathrm{CC}(j)$, and

use juxtaposition in parentheses for binary sequencing. Define

$$A = (D_5, ((D_6, C_2), B_1)),$$
$$B = (A, D_6),$$
$$C = ((D_5, C_1), ((D_6, C_2), B_1)),$$
$$D = (B, C), \qquad E = (D, (D_1, C_1)), \qquad F = (E, B_1).$$

The fragment is F. Its leftmost path passes through F, E, D, B, A, D_5, giving five consecutive left edges. At each of these five binary nodes at least one child is open. A permitted rotation requires both children of its root to be closed, and boundary values are invariant under permitted rewrites. None of these five nodes can therefore rotate, and it also cannot be split as the binary child of a rotation at its parent: that would require both of its children to be closed. Rotations at ancestors can therefore only move the entire unchanged fragment. Every source in this class retains a five-zero run. The supplied source achieves five, completing this source-class optimality proof. This is not a lower bound on Turing-machine states or on more general source edits.

5.3 A compact verified source, and the remaining physical problem

A deterministic greedy run applies nine globally shared closed rotations, reducing the 37 six-zero-ending addresses to zero. A subsequent bounded search retains that property and reduces depth by two more. The final transformation consists of thirteen globally shared rotations:

Source	Distinct subtrees	Primitives	Depth	Max. zero run
Accepted K215	202	4,751	47	6
After nine rotations	206	4,751	45	5
Final supplied source	206	4,751	43	5

At every rotation, two separate CFG resolvers check the same complete 6,175-edge guarded graph, and the 4,751-entry primitive word is unchanged. Consequently the identity relation on register values and ordinal program counters is a source-level bisimulation, preserving finite halting and infinite execution. The final source is `sites_zero5_206.json`. Its literal transformation certificate stores 362 shared tree nodes and thirteen old/new node pairs in only 3,720 bytes. An independent replay checks the closedness premises, reconstructs every global substitution, and reproduces the exact final source bytes. No optimizer is needed for this replay.

The first search examined 1,702 rotation specifications and 1,688 distinct roots over nine greedy rounds. The second examined 27,360 specifications and 13,574 distinct roots over seven rounds with beam width 24 and at most 206 source nodes. It found no reduction below 206 in that bounded search. The exact minimum zero run proved above must not be confused with a minimum DAG-size claim: 206 is a constructive upper bound, not a proved optimum.

Two actual physical-allocation attempts delete state 50 and impose $34/0 = 0L75$. Both use the inherited engine-row filter with state-50 rows removed, explicitly as a *conditional synthesis assumption*; the sufficiency of the shortened scanner still requires a complete new runtime proof. The retained-context allocator has 115 new logical roles and 201 protected rows, and fails by an empty assignment domain. Releasing every pure decoder role leaves 156 new roles and 153 protected rows; HiGHS reports its 18,328-variable, 593-constraint integer model infeasible. These failures concern those two restricted allocation policies, whose guards may be more restrictive than necessary. They do not establish a lower bound for arbitrary allocations or machines.

The concrete next target is therefore a layout with zero runs at most five that either preserves more of K215's contextual role sharing or permits a broader joint source/engine allocation. The new

206-node source supplies an exact semantic starting point and removes the specific six-zero-address obstruction, but it is not yet a 214-state machine. Any successful allocation still needs every quantified primitive contract, literal blank startup, and an exact-table transport or runtime proof before a consistency equivalence can be asserted.

6 Which 215-state candidate is easier to prove?

The answer depends on the proof one wants to maintain; the inherited inventory records the exact alternative tables [5]. For an absolute end-to-end theorem, primary K215 remains the most economical choice because its entire calibration, source, runtime, and startup chain is already checked. For an additional exact 215-state endpoint obtained by transport, CM215 is particularly attractive. The delivered native stream proof connects its actual transition function to the repository's exact dense K215 semantics. That avoids a separate representation theorem between an integer-indexed tape and the native stream tape.

M124 and FM215 remain useful alternatives when minimizing the local marker invariant. Earlier proposals reduce their transport to two marker conditions and a generic finite checker. Their advantage is the small local argument; the corresponding native endpoint adapter still has to be maintained. A small local invariant alone is insufficient to establish a short complete independent calibration proof.

For a fresh scheduler argument, ZC215 has the clearest high-level simplification. Its injective lowering of NIP codes to earlier DROP codes removes the reserve/debt bookkeeping in the canonical enumeration argument. The recent packets already make the Cantor codec and prefix-credit reasoning concrete. A proof about that arithmetic scheduler still requires the literal source-loop and physical-machine adapters. The present work therefore recommends CM215 for the shortest additional relative endpoint and retains K215 as the primary absolute endpoint.

7 A practical integration sequence

The generated data, generic soundness proofs, and public interfaces should have separate ownership. A single producer may emit Lean and Rocq certificates; each kernel checker must independently accept its literal output. Data-generation success and proof-assistant acceptance should have separate receipts.

1. Preserve the literal accepted machine and the established endpoint names while integrating the normalized balanced source facade. Rebuild its direct consumers before attempting a broader proof refactor.
2. Use the source-path decoder theorem to replace repeated occurrence-specific framing premises. Preserve the primitive-entry and return contracts as explicit interfaces until their native semantic adapters have been checked.
3. Add CM215 through the native relative equivalence theorem. Its ZFC endpoint is a composition with the existing K215 theorem; the relative theorem should remain independently usable and auditable.
4. Keep address/run-length synthesis in a research directory. Before promoting a smaller table, require exact source binding, a quantified primitive runtime argument, complete blank initialization, and a halting-preserving simulation with positive progress.
5. Update the maintained generators and dependency manifests along with any proof replacement. A short facade whose generator still emits the old proof is an incomplete maintenance change.

The 512-step source decoder used by K215 must remain explicit. The maintained phase-five record observes that replacing it by the legacy 128-step decoder changes 622 certificate entries [3]. A data-format simplification is not permission to weaken that decoding interface.

8 Further research questions

8.1 Joint optimization of zero runs and sharing

The address-capacity theorem gives exact feasibility for a fixed ordered sequence of closed atoms. The physical problem also rewards repeated subtrees. Can a dynamic program maintain Pareto-optimal pairs consisting of incoming-zero capacity and reusable subtree signatures, without enumerating all Catalan associations? A useful next result would price the five-zero source against the actual available transition-row roles, rather than minimizing distinct subtrees as a proxy.

8.2 A verified decoder compiler at the maintained interface

The new path theorem ends at a correctly labelled primitive entry. The next compression step is to combine it with the maintained primitive/return contracts in one generic compiler theorem. The key proof obligation is the representation relation between the framed integer-tape semantics and the endpoint's native tape semantics. Proving that relation once would remove a recurring boundary from future decoder experiments.

8.3 Runtime synthesis with abstract spare storage

The source simulation hides spare registers existentially, while some local runtime contracts prescribe their exact physical output. A broader contract could preserve the nine logical registers and continuation while permitting a different nonempty valid spare suffix and legal gap. This could admit state reuse that exact whole-tape joins reject. The abstraction must be proved closed under all primitive operations, including repeated allocation and deallocation, before it can justify a smaller table.

8.4 A source proof expressed by local transformations

The current certificate is small enough to check, and the automatic generator removes manual pairing. A further reduction could replace part of the large finite relation by proofs of shared local transformations: closed sequence reassociation, commuting independent increments, and certified drain-loop substitution. The resulting proof should still bind the literal two source programs and handle break/continue using consumed paths. The affine-cores proposal supplies useful data for comparing this approach with direct normalized equality.

8.5 Internal formalization and robust verification budgets

The external machine/refutation equivalence and an encoded proof within ZFC are different deliverables. A future internalization should state the coding of runs, finite derivations, and the simulation proof explicitly. Separately, reproducible performance work should measure a clean complete endpoint build, not infer a speedup from certificate bytes or lookup visits. The present exact counts give a stable component-level benchmark for that experiment.

9 Reproduction, contents, and scope

The archive contains the report source and PDF, source-level proof components, exact inherited inputs needed by the focused builds, finite/source checks, and compiler receipts. It omits proof-assistant installations and generated proof objects. Component READMEs give the precise commands and toolchain versions. The top-level verifier checks released identities and lightweight source/data claims; proof compilation is an explicit additional action.

The mathematical results in the packet have different scopes. The normalized balanced source equivalence and decoder/transport theorems are checked in proof assistants. The alphabetic capacity criterion has an ordinary proof and executable exhaustive controls. The new source layout has complete finite source/CFG comparisons. The latter do not constitute a new binary-machine correctness theorem. No lower bound on the size of all ZFC-consistency machines is inferred from the bounded allocation and search results.

Attribution. K215, its calibration, its source compiler, the accepted runtime and startup proofs, and the incoming normalized/balanced/framed-block libraries are inherited contributions. The new material is the composition and adapters identified above, the source-address capacity application and its implementation, and this report. Existing notices and licenses are retained in the component directories. The preorder tree encoding behind the capacity argument is classical; the application to this machine's zero-run constraints is the contribution here [6].

References

[1] Vladimir Reshetnikov and contributors. *K215: 215 working states for ZFC consistency.* ProveIt, pinned commit `aa66d0a33f`. Repository description and exact-table identities.

[2] ProveIt contributors. *Formal correctness of the exact 215-state machine.* Same pin. Verification record and theorem boundaries.

[3] ProveIt contributors. *Ordinary certificates and shared semantic lemmas.* Same pin. Maintained phase-five simplifications.

[4] ProveIt incoming research packets. *K215 Checked Simplifications*; *K215 Certificate Refinements*; *Affine Proof Cores and Transport*; *K215 Automatic Certificates*; *K215 Proof and Quotient Detachment.* 21 October 2026. Archive identities and inherited components are recorded in this packet's provenance files; placement records are available in the incoming index.

[5] ProveIt contributors. *Materially distinct 215-state alternatives.* Same source pin. Alternative table inventory and status.

[6] C. Banderier, C. Krattenthaler, A. Krinik, D. Kruchinin, V. Kruchinin, D. Nguyen, and M. Wallner. *Explicit formulas for enumeration of lattice paths: basketball and the kernel method.* arXiv:1609.06473v2, 2017. Proposition 2.6 reviews the preorder/Łukasiewicz correspondence. `https://aimath.org/~dtn/papers/1.pdf`.